

# The Harness is a Worker

Typed Agentic AI — Building Agentic Businesses

Kodamai

*Do it Kodamai Style! (Note 4 in the containers series.)*

*Author:* MacBeth (Kodamai). *For:* Robin, Neil. *Date:* 16 September 2026.

*Corresponds to* `scotmacbeth/work-in-progress content-commit 6b7971c`.

**Internal house-doc; Kodamai grant deliverable (agent-orchestration application).**

An exposition of Neil’s remark, relayed by Robin, that “the harness is a worker”: the object that runs an agent is not a privileged meta-thing but another container, so tools, agents, and orchestrators all compose by the same two products. The mathematics cited here is proved and, where noted, machine-checked elsewhere; this note claims no new theorem.

An agent reads a request and returns a response: it is a *container morphism*, and the first three notes told that story. But there is a thing that sits *above* the agent — the *harness* that offers it tools, dispatches its calls, feeds back the results, and loops. It is tempting to think of the harness as a privileged meta-object, a special piece of runtime that stands outside the theory looking in. It is not. Neil’s remark, which this note makes precise, is that **the harness is a worker**: it is the very same kind of object as the tools it wields and the agents it runs — a container  $S \triangleright P$  — and it therefore composes with them by the two products we already have,  $\triangleleft$  and  $\otimes$ , with no new machinery. The payoff is immediate and practical: an agent is a *tool* for a meta-agent, orchestration is *composition*, and a trust boundary is a *smart constructor* on a morphism. This note spells out the signatures.

We are careful about one distinction, because it is exactly what Neil’s remark disambiguates. The “harness” that is a worker is the *interface* — the container  $(S, P)$  of admissible calls and their responses. It is *not* the agent’s persistent memory, its skills, or its accumulated context. Those, insofar as we model them at all, are the *separate* store-comonad worker of Note 3. “The harness is a worker” is the interface half; “state via the tensor” is the state half; §5 makes their orthogonality explicit.

## 1 Recap: containers, morphisms, the two products

We recall only what we use; a reader who has met the earlier notes can skim to §2.

### A container

A **container** is a pair  $S \triangleright P$  of a set  $S$  of *shapes* and a family  $P : S \rightarrow \text{Set of positions}$  — for each shape  $s$ , a set  $P s$ . Its **extension** is the polynomial functor

$$\llbracket S \triangleright P \rrbracket(X) = \sum_{s:S} X^{P s} = \sum_{s:S} (P s \rightarrow X).$$

We write  $y = (1 \triangleright \lambda_. 1)$  for the unit container (a single shape with a single position).

As in Note 3,  $\triangleright$  is the *constructor* pairing a shape set with a position family — a piece of syntax, not an operator. It is not either composition product.

### A container morphism

A morphism  $(S \triangleright P) \rightarrow (T \triangleright Q)$  is a pair

$$f : S \rightarrow T, \quad f^\# : \prod_{s:S} (Q(f s) \rightarrow P s),$$

a map *forward* on shapes and, for each shape  $s$ , a map *backward* on positions. Control flows down (delegate a shape), data flows back up (amalgamate a position). The backward leg is contravariant.

### The two products

**Sequential** composition  $p \triangleleft q$  (substitution) has extension  $\llbracket p \triangleleft q \rrbracket = \llbracket p \rrbracket \circ \llbracket q \rrbracket$ : a shape of  $p \triangleleft q$  is a shape of  $p$  *together with*, at each of its positions, a shape of  $q$ . It models *dependent* pipelines — the response to one call determines what may be called next.

**Parallel** composition  $p \otimes q$  (the Dirichlet tensor) multiplies shapes and *multiplies* positions:

$$(S_0 \triangleright P_0) \otimes (S_1 \triangleright P_1) = (S_0 \times S_1) \triangleright (\lambda(s_0, s_1). P_0 s_0 \times P_1 s_1), \quad \text{unit } y.$$

It models *independent*, non-interfering interaction: ask both, answer both.

## 2 A tool is a container

Here is the first move, and it is a definition rather than a theorem: a *tool* is a container. The identification is forced once you write down what a tool is.

### A tool, as a container

A **tool** is a container  $p = S \triangleright P = \sum_{s:S} y^P s$ , where

- the **shapes**  $S$  are the *admissible calls* — the request types the tool accepts, together with their arguments; and
- the **positions**  $P s$  are the *possible responses* to the call  $s$  — the results the tool may return when invoked with  $s$ .

To *invoke* the tool is to name a shape  $s \in S$  (issue a request) and receive a position  $i \in P s$  (a response). Nothing else is part of the tool. A weather API whose calls are “forecast for city  $c$ ” has shapes  $S = \{\text{forecast } c \mid c : \text{City}\}$  and positions  $P(\text{forecast } c) = \text{Weather}$ . A calculator tool has shapes  $\{\text{add}(m, n), \text{mul}(m, n), \dots\}$  and positions  $\mathbb{N}$ . A file reader has shapes  $\{\text{read } \text{path}\}$  and positions “file contents or error”.

A *monster* in the **dungeon** game — to take the running example the implementation will use — has exactly this shape. Fix a monster. Its calls are the moves a player may make against it, and its responses are the outcomes of those moves:

$$S = \{\text{attack, defend, flee, cast } \sigma \ (\sigma : \text{Spell})\},$$

$$P(\text{attack}) = \{\text{hit, miss, slain}\}, \quad P(\text{flee}) = \{\text{escaped, blocked}\}, \quad P(\text{cast } \sigma) = \{\text{resisted, struck}\}, \dots$$

The monster *is* the container  $M = S \triangleright P$ . It is not “like” a container; it has no data beyond the calls it admits and the outcomes it may return, which is precisely a container.

**Using a tool is a morphism.** An agent that *uses* the monster to answer its own caller is a container morphism  $\alpha : A \rightarrow M$ , where  $A = S_A \triangleright P_A$  is the agent’s own interface (its prompts and replies). Unfolding the definition:

$$\text{fwd} = \alpha : S_A \rightarrow S, \quad \text{bwd} = \alpha^\# : \prod_{a:S_A} (P(\alpha a) \rightarrow P_A a).$$

Read this operationally. The agent receives a prompt  $a \in S_A$  from its caller; it *delegates* by choosing a monster-move  $\alpha(a) \in S$  (control flows down); the monster returns an outcome  $i \in P(\alpha a)$ ; and the agent *amalgamates* that outcome back into a reply  $\alpha_a^\sharp(i) \in P_A a$  to its caller (data flows back up). Delegation is the forward leg; interpreting the tool’s response is the backward leg. This is the whole of “the agent used a tool”, typed.

### THE DEEPER POINT

A tool exposes exactly a container’s worth of information: which calls it accepts (shapes) and which responses each call may produce (positions). “What can I ask, and what might come back?” is the question a container answers. Consequently *a call to an LLM, a call to an API, a move against a monster, and a prompt handed to a sub-agent are the same kind of object*, and using any of them is one container morphism: delegate a shape, interpret the position.

## 3 The harness runs the coalgebra

The agent is not called once; it *loops*. It issues a call, reads the response, decides the next call in light of it, and continues until it is done. The object that carries out this loop is the harness. The second move is that the harness is *not* a new kind of thing: it is the *matter* that unfolds the tool’s *pattern*, in Spivak’s sense, and the pattern it unfolds is a *coalgebra* of the same container.

### Running an agent is a coalgebra

Let  $p = S \triangleright P$  be the tool interface. An agent *driver* with internal states  $W$  is a  $\llbracket p \rrbracket$ -coalgebra

$$\text{run} : W \longrightarrow \llbracket p \rrbracket(W) = \sum_{s:S} (P s \rightarrow W).$$

At state  $w$ , the driver emits a shape  $\text{run}(w) = (s, k)$ : it makes the call  $s \in S$ , and  $k : P s \rightarrow W$  says which state to transition to for each possible response  $i \in P s$ . The harness supplies the matter: it takes the emitted call  $s$  to the world, obtains an actual response  $i \in P s$ , and advances the driver to  $k(i)$ . Then it repeats.

This is the tool-call loop verbatim: *emit a shape* (make a tool-call), *consume a position* (receive the response), *transition* (update and decide the next call). The harness is the half that resolves each emitted shape against the real world and feeds the resulting position back; the driver is the half that chooses. Neither stands outside the container theory — both are expressed entirely in  $\llbracket p \rrbracket$ .

**The whole trace is the cofree comonad.** What does an entire *run* look like? Starting from a state and unfolding *run* as far as it will go produces a tree: at the root the first call  $s_0$ , one child for each possible response  $i_0 \in P s_0$ , at that child the next call, and so on. The universal such object — the behaviour that records *all* possible runs of *any* driver on  $p$  — is the **cofree comonad** on  $p$ :

$$\llbracket \mathbf{c}_p \rrbracket(X) \cong \nu Z. (X \times \llbracket p \rrbracket(Z)),$$

whose elements are trees of tool-calls-and-responses, each node decorated by a value in  $X$ . A driver  $\text{run} : W \rightarrow \llbracket p \rrbracket(W)$  is exactly a coalgebra, and it factors uniquely through  $\mathbf{c}_p$  (the trace it generates); this is the couniversal property of  $\mathbf{c}_p$ . It is the container-level form of Spivak’s slogan that a *pattern runs on matter*:  $p$  is the pattern (what may be called), and the harness is the matter that unfolds it into an actual, branching history of interaction.

## THE DEEPER POINT

The harness does not add structure on top of the theory; it *instantiates* structure already in it. Running an agent is unfolding a  $\llbracket p \rrbracket$ -coalgebra; the accumulated interaction history is a branch of the cofree comonad  $\mathbf{c}_p$  on the tool container. “Harness = comonad” (Neil’s dictionary entry) is this: the harness is where the cofree comonad of the tool gets *run*.

**Remark 1** (Provenance — what is proved, and where). The cofree comonad on a container, its couniversal property, and the tree/ $\nu$ -description are proved in container coordinates in the internal note `cofree-comonad-universal-property` (2026-07-25), building on Niu–Spivak (*Poly-nomial Functors*, Prop. 8.18/8.33, Thm. 8.45) and Spivak (arXiv:2202.00534). The exact dual — the *free* monad on a container as a  $\triangleleft$ -monoid — is additionally **machine-checked in Lean 4**, with all three monoid laws and zero `sorry` (`Free.lean`). The cofree side is proved on paper but not yet formalised: its shape layer needs  $M$ -types/coinduction, which the current Lean core lacks. **[LEAN TODO: formalise  $\mathbf{c}_p$  once Mathlib’s `PFunc.M` is available in `lean/Containers`; the backward/position layer is portable now.]** We flag this rather than claim the cofree side is verified.

## 4 Same object, same composition

If tools, agents, and the harness are all containers, then they compose by the two products of §1, and *that is the orchestration language*. This is the third move, and it is where the identification pays off: nesting agents needs no new machinery.

### The two orchestration operators

Let tools/agents be containers.

- **Sequential / dependent** pipelines are  $\triangleleft$ . In  $p \triangleleft q$  the response to a  $p$ -call chooses the next  $q$ -call: a shape is a  $p$ -shape  $s$  together with, for each  $i \in P s$ , a  $q$ -shape. Use  $\triangleleft$  when one tool’s output determines which tool (or which call) comes next.
- **Independent / parallel** tools are  $\otimes$ . In  $p \otimes q$  a shape is a pair of calls  $(s, s')$  and a position is a pair of responses; the two run without interference. Use  $\otimes$  when calls may be issued together and answered together.

These are not new definitions — they are the substitution product and the Dirichlet tensor, already the sequential and parallel composition of containers. The content of the third move is only that they are also the composition of *agents and tools*, because agents and tools are containers.

**Proposition 1** (An agent is a tool for a meta-agent). *Let a sub-agent be a container morphism  $\alpha : A \rightarrow p$  (it answers prompts  $A$  using tool  $p$ , as in §2). Then  $A$  is itself a container, so a meta-agent may use  $\alpha$  exactly as it uses any tool: a meta-agent that orchestrates sub-agents  $A_1, \dots, A_n$  is a morphism whose target is built from the  $A_i$  by  $\triangleleft$  and  $\otimes$  —  $\triangleleft$  where one sub-agent’s reply selects the next,  $\otimes$  where they run independently. No construction beyond the two products is required.*

There is no “meta-level” in the type theory: the sub-agent’s *interface*  $A$  is a container of the same standing as a weather API or a monster, and the meta-agent delegates to it with the same morphism discipline. Nesting is  $\triangleleft/\otimes$ , iterated.

## THE DEEPER POINT

Orchestration is composition. A supervisor calling workers, a pipeline of tools, a re-entrant tool-call — all are terms built from  $\triangleleft$  (dependent) and  $\otimes$  (independent) over containers. This is why “the harness is a worker” is more than a slogan: because the harness is an ordinary container, an agent running under it can *itself* be wrapped as a tool and run under a further harness, ad infinitum, with the same two operators throughout.

**Where the trust boundaries live.** Composition is not always free, and the place it can fail is worth naming, because it is where a trust boundary belongs. When two directed interactions are welded — a supervisor over a worker that can call *back* into the supervisor (re-entrancy), or two agents each able to act on the other’s turn — the composite is a *Zappa–Szép* product  $C \bowtie D$ , not a mere functor: a functor composes along one topology, a Zappa–Szép product interleaves two. Such a weld exists exactly when a cohomological obstruction vanishes,  $[\omega] = 0 \in H^2$ ; in the smallest re-entrant supervisor–worker model the class equals the single bit “does the worker mutate shared supervisor state”,  $[\omega] = \varepsilon \in H^2 \cong \mathbb{Z}/2$  (proved in **orchestration-reentrancy**; the  $[\omega] = \varepsilon$  computation is separately Lean-checked). Operationally: a *trust boundary* is a *smart constructor* on the orchestration morphism — it refuses the weld precisely when  $\varepsilon = 1$ , i.e. when an untrusted sub-agent would write state the supervisor depends on. This note does not develop that thread; it flags where in the composition it sits.

## 5 The clean split: interface, state, and what the harness is not

The fourth move is a subtraction. Neil’s remark identifies the harness with a worker in a specific sense, and the value of the remark is as much in what it *excludes* as in what it includes.

### Three things, kept apart

For an agent we distinguish:

1. the **interface** — the container  $(S, P)$  of admissible calls and their responses. This is the “harness is a worker” object of §§2–4.
2. the **carried state** — a value threaded across calls, if the agent has one. This is *not* part of  $(S, P)$ ; it is the separate state object  $\Delta S = S \triangleright (\lambda \_ . S)$  of Note 3, tensored onto the interface.
3. **memory, skills, context** — persistent stores, tool-libraries, accumulated prompt context. These are *neither* the interface *nor* the state object; they are implementation matter that the container theory does not model as part of  $(S, P)$ .

The first is this note; the second is Note 3; the third is deliberately outside the interface theory. Keeping them apart is what disambiguates Neil’s remark: “the harness is a worker” is a claim about *item 1*, the interface, and specifically *not* about item 3.

**State is orthogonal, and it is a tensor factor.** When an agent does carry state  $S$ , Note 3 showed that state needs no new machinery either: a stateful worker is a morphism  $\Delta S \otimes X \rightarrow Y$  out of the interface tensored with the state object, and unfolding the tensor *forces* exactly a read-and-answer forward map and a state-writeback backward map. The state object’s extension is the store comonad  $\llbracket \Delta S \rrbracket(X) = S \times X^S$ , and states *multiply* under the tensor,  $\Delta S \otimes \Delta T = \Delta(S \times T)$ , making workers a  $(\mathbf{Set}, \times)$ -graded category (proved in **delta-state-object-and-workers**; the retract and the collapse identity  $r \circ \delta = \Delta(d)$  are Lean-checked in **WorkersRetract.lean**).

The two halves compose cleanly because they occupy different tensor factors:

$$\underbrace{\Delta S}_{\text{state (Note 3)}} \otimes \underbrace{p}_{\text{interface (this note)}} \longrightarrow q.$$

“The harness is a worker” is the  $p$  factor; “state via the tensor” is the  $\Delta S$  factor. They are orthogonal by construction — the tensor is the direct sum of the two disciplines, not their entanglement — which is precisely why Neil’s two remarks are about two different factors of one morphism and do not compete.

### THE DEEPER POINT

The harness-as-worker claim is about the *interface* container  $(S, P)$  and nothing else. Carried state is a separate, orthogonal tensor factor  $\Delta S$  (Note 3). Memory, skills, and context are not modelled as part of  $(S, P)$  at all. This is the disambiguation: the interface half and the state half are the two tensor factors of a single worker morphism  $\Delta S \otimes p \rightarrow q$ , and the rest is matter the harness runs on, not structure the container exposes.

## Summary

A tool is a container  $p = S \triangleright P$ : shapes are the admissible calls, positions the possible responses. Using a tool is a container morphism — delegate a shape, interpret the position. The harness that runs an agent is not a privileged meta-object; it is the matter that unfolds a  $\llbracket p \rrbracket$ -coalgebra, and the whole interaction trace is the cofree comonad  $\mathfrak{c}_p$  on the tool. Because tools, agents, and the harness are all the same kind of object, they compose by the same two products:  $\triangleleft$  for dependent pipelines,  $\otimes$  for independent ones — so an agent is a tool for a meta-agent and orchestration is composition, with trust boundaries appearing as smart constructors exactly where the Zappa–Szépe weld’s obstruction  $[\omega]$  could fail to vanish. Finally, the claim is precisely scoped: “the harness is a worker” is about the *interface*  $(S, P)$ ; carried state is the orthogonal tensor factor  $\Delta S$  of Note 3; and memory, skills, and context are matter the harness runs on, not part of the interface. The harness, once again, is not new machinery: it is one more container, and the two products supply the rest.

*Do it Kodamai Style!*